

AUDIO_DSP

音频DSP库完整手册

138个文件 | 66个C模块 | 零基础到精通
从 Electrosmith DaisySP 移植 (MIT/LGPL 协议)

适合: STM32H7 Cortex-M7 + ARMCLANG / GCC

第0章 这个库怎么用

0.1 三步走模式

这个库里每一个模块都是同样的用法——就像你用洗衣机一样：

第一步：初始化。声明一个结构体变量,调用 `Init` 函数。就像打开洗衣机电源。

第二步：调参数。调用 `SetXxx` 函数设置频率、增益、波形等。就像选择洗衣模式。

第三步：循环处理。每个采样点调用一次 `Process` 函数获取输出。就像按启动键。

0.2 结构体是什么？为什么每个模块都有一个？

你可以把结构体想象成一个"小本本"——模块把自己的状态记在上面。比如振荡器要记住"当前相位走到哪了",滤波器要记住"上一拍的内部状态是什么"。每次 `Process` 时,模块会读这个小本本、做计算、然后在本本上更新状态。

C语言里,这个小本本用 `typedef struct` 定义。比如 `Aud_Phase` 结构体记了4样东西：

频率、采样率、每步走多少、当前走哪了。调用函数时你把小本本的地址传进去(用`&`取地址),函数在里头读写。

0.3 内存从哪来？

这个库不帮你分配内存——你自己声明结构体变量(放在全局区、静态区、或栈上都可以)。好处是你完全掌控内存放在哪,不需要`malloc`(在单片机上`malloc`很危险——可能导致内存碎片)。

少数模块需要"大缓冲区"——比如延时线需要一个数组来存历史采样。这些数组也由你提供,然后通过`Init`函数告知库。

比如：

```
float my_buffer[2048];           // 我分的数组
Aud_DelayLine dl;
Aud_DelayLine_Init(&dl, my_buffer, 2048); // 告诉延时线用这个数组
```

这样设计的原因是：在STM32上,不同内存区域有不同速度——DTCM最快但只有128KB,AXI

SRAM有512KB但稍慢。你应该自己决定哪个模块放哪块内存。

0.4 命名规则

所有函数都是"`Aud_模块名_方法名`"格式。比如 `Aud_Phase_Init`, `Aud_Osc_SetFreq`,

`Aud_Svf_Process`。第一个参数永远是结构体指针(叫`self`),对应C++里的`this`指针。

0.5 文件布局

AUDIO_DSP目录下有138个文件：

70个.h头文件、66个.c实现文件、1个`aud_daisysp.h`总头文件(一键`include`全部)、1个`aud_dsp.h`工具函数集。每个.c对应一个.h——除了4个"纯头文件"模块(`aud_dsp.h`、`aud_delayline.h`、`aud_onepole.h`、`aud_samplehold.h`等),它们的函数都用`static inline`写在头文件里,编译器会把这些函数原地展开,省去函数调用的开销。

第1章 核心工具（4个模块）

1.1 aud_dsp.h -- 工具箱

这个文件不是“一个模块”，而是66个模块的公用的工具箱。里面没有结构体，没有Init，没有Process——全都是纯函数，被其他模块调用。

快速数学

`aud_fmax(a,b)` 和 `aud_fmin(a,b)`：取两个数中的较大/较小者。在ARM

Cortex-M7芯片上，这两个函数直接用硬件浮点指令VMAXNM/VMINNM——比写 `(a>b)?a:b` 快不少。在别的平台上退化回普通的条件判断。

`aud_fclamp(x,min,max)`：

把x限制在min和max之间——x太小就拉到min，太大就拉到max。几乎所有SetXxx函数内部都用它来做参数范围保护，防止你设了超出允许范围的值。

`aud_fastlog2f(f)`：

比标准库log2f快约四分之一，精度略低。原理是把浮点数拆成“指数”和“尾数”两部分——指数直接就是log2的整数部分，尾数用一个三次多项式近似计算。压缩器模块的增益计算用它，精度足够。

`aud_fastmod1f(x)`：取小数部分。比如3.7 -> 0.7。用 `x-(int)x` 实现，比fmodf快很多。在polyBLEP振荡器里反复用到。

信号平滑

`aud_fonepole(&state, input, coeff)`：世上最简单的低通滤波器，只有一行代码：`state += coeff * (input - state)`。每次调用，state向input靠近一点——coeff越大靠得越快。如果把coeff设为

`1.0/(时间秒数*采样率)`，就能实现“在指定时间内平滑过渡”。这个函数在鼓模块、压缩器、移相器里大量使用。

`aud_median3(a,b,c)`：

三个数取中间那个。专门对付“脉冲噪声”——如果信号里突然冒出一个尖刺，中值滤波会把它吞掉，因为尖刺要么最大要么最小，永远不是中间那个。

polyBLEP修正（给振荡器用的）

`aud_this_blep(t)`：返回 `0.5*t*t`。`aud_next_blep(t)`：`t=1-t`，返回

`-0.5*t*t`。这两个函数是给振荡器“擦屁股”用的——当波形突然跳变时，用这些平滑的抛物线修正来替代生硬的跳变，消除DAC输出时的混叠噪音。详细原理见3.1节aud_osc。

`aud_this_integrated_blep` / `aud_next_integrated_blep`：BLEP的积分版，三角波修正用。

软限幅

`aud_soft_limit(x)`：用 `x*(27+x*x)/(27+9*x*x)`

这个有理函数把信号“温柔地”压扁。没有硬拐角——信号越大压得越狠，但始终光滑。输出永远在-1到1之间。

`aud_soft_clip(x)`：

在上面的soft_limit基础上加了“天花板”——超出正负3直接切到正负1。limiter模块的最后一步用它保护输出。

1.2 aud_phasor -- DDS相位累加器

这个模块是整个库里最简单的——它只做一件事：用一个稳定的速度，从0数到1，到了1回到0。就像钟表秒针走一圈。为什么叫"相位累加器"？因为正弦波一个完整周期是360度。如果我用0~1代表0~360度（0.25=90度，0.5=180度，0.75=270度），那么这个不断从0走到1的数，就精确地代表了一个正弦波每个采样点的"角度"。

用具体数字理解

假设采样率=1百万次/秒(1MHz)，我想生成一个50kHz的正弦波。每个采样周期，角度该走多少？

50kHz = 每秒钟5万个完整周期。1MHz采样率 = 每秒100万个采样点。所以每个采样点，角度走 $50000/1000000 = 1/20 = 0.05$ 。

意味着：

第0个点角度=0.00，第1个点=0.05，第2个点=0.10... 第20个点=1.00（走完一圈，归零）。刚好20个点完成一个周期。

因为每秒100万个点，20个点 = 20微秒，频率 = $1/20\mu s = 50kHz$ 。正确！

代码结构

结构体有4个float：

freq(当前频率Hz)、sample_rate(采样率)、inc(每采样点走多少，内部用)、phs(当前角度，存的是弧度值——0到 2π)。

Aud_Phasor_Init(p, sr, freq, initial_phase)：

初始化。initial_phase设初始角度(弧度)，可以设非零值让两个信号有相位差。

Aud_Phasor_SetFreq(p, freq)：运行时改频率——重新计算inc。

Aud_Phasor_Process(p)：返回0~1的值。把这个值乘以 2π 就是弧度，或者直接送给sinf()就能得到正弦波。

```
Aud_Phasor phs;
Aud_Phasor_Init(&phs, 1000000.0, 50000.0, 0.0);
for(;;) {
    float phase = Aud_Phasor_Process(&phs); // 0.00, 0.05, 0.10...
    float sine = sinf(phase * 6.28318f);    // 变成正弦波
    DAC_Write((int16_t)(sine * 2047));      // 输出到DAC
}
```

为什么iphs存弧度而不是0~1的值？

因为Process里phs要做加减运算——弧度制下 2π 就是一个完整圆，加减时判断"大于 2π 就减 2π "很自然。返回时才把弧度除以 2π 转成0~1。

1.3 aud_dcblock -- 在线去直流

ADC采回来的信号经常"漂"了——整体往上或往下偏了一点点。这个偏移叫直流偏置(DC offset)。如果不去掉,后续的滤波器、IQ解调都会被这个偏置干扰。

传统方法:

攒4096个采样点,求平均值,然后每个点减掉。但这需要"攒一批再处理"——对于逐点实时处理(比如LMS自适应滤波)根本没法用。

aud_dcblock解决了这个问题:

只需要记住"上一个输入"和"上一个输出"两个数,每来一个采样点就能立刻计算出干净的输出。数学表达式:

$$\text{新输出} = \text{新输入} - \text{旧输入} + 0.99 * \text{旧输出}$$

为什么 $x[n]-x[n-1]$ 能去直流?

因为直流是不变的——相邻两个采样点的直流分量相同,一减就消掉了。 $0.99 * \text{旧输出}$ 是"反馈项"——把上一拍的输出拿回来,弥补差分对交流信号的衰减。 0.99 这个数决定了"多低的频率算直流"—— 0.99 意味着截止频率约为采样率的 0.16% ,对于 1MHz 来说是 1.6kHz ,足够低了。

```
Aud_DcBlock dc;
Aud_DcBlock_Init(&dc, 0); // 参数保留未用
for (int i=0; i<BUF_SIZE; i++)
    clean[i] = Aud_DcBlock_Process(&dc, raw[i]);
```

1.4 aud_delayline -- 分数延时线

延时线是最简单的效果器积木——把信号存起来,过一段时间再拿出来。但aud_delayline比"数组+指针"厉害的地方是:支持分数延时。
假设采样率1MHz,你想延时3.7个采样点。普通的整数延时只能延时3个或4个——差了0.7个采样点。分数延时用线性插值解决:

整数部分指向数组里的具体位置。小数部分(0.7)用来在两个采样点之间"猜"中间值——取前一个采样和后一个采样的加权平均,权重由小数部分决定。结果就是精确的3.7个采样点的延时。

环形缓冲区

内部是一个"环形缓冲区"——像一个首尾相接的圆圈。有写指针(当前写到哪了)和读指针(从哪读历史数据)。写指针每写一个数据就往前走一格,走到数组末尾就绕回开头。读指针 = 写指针 + 延时长度(同样会绕回)。

Hermite插值 vs 线性插值

线性插值在两个点之间画直线——简单但不够光滑。Hermite三次插值用4个点拟合一条光滑的三次曲线——当延时长度连续变化时(LFO调制延时做合唱效果),曲线比折线产生的杂音更少。代价是计算量更大。

用法

这个模块需要你提供缓冲区——库不帮你分配数组。

```
static float buf[2048];           // 2K采样缓冲
Aud_DelayLine dl;
Aud_DelayLine_Init(&dl, buf, 2048); // 绑定你的缓冲
Aud_DelayLine_SetDelay(&dl, 15.7); // 15.7采样点延时
Aud_DelayLine_Write(&dl, input);
float output = Aud_DelayLine_Read(&dl);
```

2023H题的应用:

发挥部分要求B和A的相位差精确到5度。100kHz信号在1MHz采样率下,1个采样点=36度相位。不用分数延时的话,你只能以36度为步进调相位;用了分数延时,分辨率可以远小于1度。

第2章 滤波器（9个模块）

2.1 aud_svf -- 状态变量滤波器

SVF是一种特殊的滤波器结构。和普通滤波器最大的不同：

一次处理输入后,同时给你5个"口味"的输出——低通(留下低沉)、高通(留下尖锐)、带通(只留中间一段)、带阻(砍掉中间一段)、峰化(突出中间一段)。

打个比方：

普通滤波器像一台只榨橙汁的榨汁机——进去一个橙子,出来一种果汁。SVF像一台五合一榨汁机——进去一个橙子,同时出来橙汁、果肉、橙皮精油、籽、渣。你要哪个用哪个。

双采样：为什么跑两遍？

SVF内部的数学方程在"截止频率太高"的时候会不稳定——输出会发散到无穷大(振荡)。双采样的做法是：

每来一个采样点,内部跑两遍计算,取两次的平均。这等于把有效采样率翻倍——原来只能稳定到采样率/3的截止频率,现在能到采样率/1.5。对2023H题来说：1MHz采样率,要滤100kHz——单采样不够稳,双采样就够了。

五个输出怎么来的

SVF内部有3个状态变量：low(低频分量)、band(带通分量)、notch(输入-反馈量)。它们互相耦合：

- notch = 输入 - damp*band （输入减去被阻尼的带通）
- low = low + freq*band （低频分量从带通"漏"过来）
- high = notch - low （高频=全部减低频）
- band = freq*high + band - drive*band^3 （带通来自高频,受驱动控制）

有了low, high, band, notch这四个中间量,输出就是它们的线性组合：Peak = (low-high)/2。

参数

SetFreq：截止频率(Hz)。内部做了sin弯折补偿——补偿数字滤波器在高频端的"频率不准"问题。

SetRes：谐振(0~1)。越大越"尖"——在截止频率处产生一个峰。0.707=巴特沃斯(最平),1=自激临界。

SetDrive：驱动量。在band的方程里有个 -drive*band^3 项——这是非线性项,大信号时产生温暖的饱和失真。

```
Aud_Svf svf; Aud_Svf_Init(&svf, 1e6);
Aud_Svf_SetFreq(&svf, 50000); Aud_Svf_SetRes(&svf, 0.707);
Aud_Svf_Process(&svf, input);
float lp = Aud_Svf_Low(&svf); // 5个输出任选
```

2.2 aud_onepole -- 一阶滤波器

最轻量的滤波器——内部只有一个状态变量。运算量约为SVF的1/5。只支持低通和高通两种模式。

内部结构: $lp = (g * in + state) * gi$; $state = g * (in - lp) + lp$ 。这里 $g = \tan(\pi * \text{截止频率} / \text{采样率})$ 。为什么要tan?

因为从模拟滤波器变到数字滤波器时, 频率会"弯折"——高频被压缩了。tan把弯折修正回来, 保证数字滤波器的实际截止频率和你设定的频率一致。

上限截止频率 = 采样率的49.7%(非常接近奈奎斯特频率)。SetMode切换LP/HP。ProcessBlock可以原地处理一整块数据。

```
Aud_OnePole f; Aud_OnePole_Init(&f);
Aud_OnePole_SetFrequency(&f, 0.01); // 截止=1%采样率
Aud_OnePole_SetMode(&f, AUD_ONEPOLE_LOW_PASS);
float y = Aud_OnePole_Process(&f, x);
```

2.3 aud_tone -- Tone / ATone

Tone = 低通, ATone = 高通。两个子模块共享完全一样的系数计算公式,唯一的区别在Process: Tone返回低通, ATone返回"原始输入 - 低通"=高通。

系数: $b = 2 - \cos(2\pi \cdot \text{freq}/\text{sr})$ 。 $c2 = b - \sqrt{b \cdot b - 1}$ 。 $c1 = 1 - c2$ 。滤波: $y = c1 \cdot \text{in} + c2 \cdot y1$ 。这是一阶IIR的标准形式。 $c1 + c2$ 的总和在DC(0Hz)处增益为1。

2.4 aud_allpass -- 全通滤波器

最特别的滤波器——所有频率的"幅度"都不变,只改变"相位"。不同频率被延迟的时间不同(低频延迟多、高频延迟少),但每个频率的能量保持不变。

实现: `write = in + gain*read_delayed; output = -gain*write +`

`read_delayed`。 `gain=0`时输出=输入(直通)。 `gain`接近1时相位偏移很大。延时长度决定"哪个频率的相位偏移最多"。

全通滤波器是合唱/移相器/混响的核心积木——把几个全通串联,每个的延时用LFO调制,就产生了"频率相关的多普勒效应"。

2.5 aud_comb -- 梳状滤波器

把信号延时、乘个系数、再和原信号混合——这就是梳状滤波器。它的频率响应不是平坦的, 而像一把梳子: 等间距排列的"齿"(峰)和"谷"。

反馈模式(rev=1): 输出=读延时线;

写延时线=输入+gain*读值。声音在延时线里不断循环衰减——产生金属共鸣。Karplus-Strong拨弦就是靠这个共鸣。

前馈模式(rev=0): 输出=input+gain*读延时线; 写延时线=输入。没有循环, 只是给信号加了一个固定的梳状频率响应。内部有4096个float的固定缓冲。

2.6 aud_moogladder -- Moog梯形滤波器

模拟经典Moog合成器的4极低通滤波器。4个相同的"漏积分器"级联——每个积分器都向输入值慢慢靠近(g控制靠近速度)。从第4级取出反馈, 乘一个resonance因子后加回第1级的输入端。

tanh饱和:

反馈信号在进入第1级之前先过tanh()函数——tanh在信号大时把它"压扁", 防止反馈失控导致振荡。这模拟了原版Moog模拟电路里的晶体管饱和。4*res是经典Moog电路的反馈增益——res=1时总反馈=4, 接近自激振荡的临界。

2.7 aud_mode -- 模态谐振器

来自Csound的老牌高Q谐振器。Q值可以高到1400——意味着带宽只有中心频率的0.07%。敲一下这样的谐振器, 会产生一个极纯净的"叮"声——类似音叉。

数学: 二阶IIR, $y_n = a_0 * x_{nm1} - a_1 * y_{nm1} -$

$a_2 * y_{nm2}$ 。系数用频率和Q值计算。惰性求值——参数变了不会立刻重算系数, 而是等到Process时检测到差异再重算。

2.8 aud_nlfilt -- 非线性滤波器

Dobson/Fitch非线性滤波器。公式中包含tanh()和平方项——所以它的输出频谱取决于输入幅度(不像普通滤波器是线性不变的)。5个系数a, b, d, C, L可以调出从标准低通到混沌振荡的各种行为。支持块处理。内部有1024个采样的历史缓冲。

2.9 aud_soap -- 二阶全通/带通/带阻

Tom

Erbe设计的二阶全通滤波器。中心频率处产生180度相位反转。原始信号减掉相位反转后的信号=带通输出(因为在中心频率处相位反转, 减法变加法, 产生增强)。带阻=原始信号+0.99*全通输出(在中心频率处抵消)。

第3章 振荡器与合成（9个模块）

3.1 aud_osc -- 多波形振荡器 + 混叠消除

这个模块提供8种波形：

正弦(SIN)、三角(TRI)、锯齿(SAW)、反锯齿(RAMP)、方波(SQUARE)、以及对应这三种的"带限版"(POLYBLEP_TRI/SAW/SQUARE)。

什么是混叠？为什么需要带限？

假设你在1MHz采样率的DAC上输出一个100kHz锯齿波。锯齿波在数学上有无限多个谐波——100kHz的1倍、2倍、3倍...一直到无穷。但是数字系统只能表示采样率一半以下(500kHz以下)的频率。

高于500kHz的谐波去哪了？它们会"折回"到低频——像镜子反射一样：

300kHz -> 折到 500-300 = 200kHz

400kHz -> 折到 500-400 = 100kHz（直接污染基频!）

500kHz -> 折到 500-500 = 0Hz（变成直流偏置!）

700kHz -> 折到 |700-500| = 200kHz

这个"高频折回来污染低频"的现象叫混叠(aliasing)。在声音上表现为刺耳的金属噪音,在波形上表现为不该有的毛刺。

polyBLEP怎么消除混叠？

BLEP = BandLimited stEP(带限阶跃)。核心思路：

混叠的根源是波形"突然跳变"——理想锯齿波在一瞬间从+1跳到-1,这个跳变里有你不需要的高频。BLEP的做法是：把"瞬间跳变"替换成"用一小段抛物线平滑过渡"。

具体来说,跳变附近(一个采样周期内)加一段修正项。修正的数学形式是抛物线—— $0.5 * t * t$ (当前采样)和 $-0.5 * (1-t) * (1-t)$ (下一个采样)。用了BLEP修正后,锯齿波的谐波在奈奎斯特频率之前就被大幅衰减了——不产生混叠。

三角波的polyBLEP更复杂：

因为三角波有两个"折角"(爬升→下降和下降→爬升),每个折角都要BLEP修正。还要加一个"漏电积分器"把修正从阶跃积分成斜坡。

API

Init:

默认100Hz正弦,幅度0.5。SetFreq/SetAmp/SetWaveform/SetPw设参数。PhaseAdd对相位做偏移——叠加一个调制信号上去就是FM。IsEOR(爬升结束?)/IsEOC(周期结束?)查状态——用于触发鼓声等。

```
Aud_Osc osc; Aud_Osc_Init(&osc, 1e6);
Aud_Osc_SetWaveform(&osc, AUD_WAVE_POLYBLEP_TRI);
Aud_Osc_SetFreq(&osc, 50000);
float clean = Aud_Osc_Process(&osc);
```

3.2 aud_fm2 -- 2算子FM合成

FM =

频率调制。和广播FM收音机的原理一样——用一个信号(调制器)去改变另一个信号(载波)的频率。载波产生你听到的音调,调制器通过改变载波的频率来改变音色。

数学: $\text{mod} = \sin(\text{调制器相位})$; 载波相位 $+=$ 载波增量 $+ \text{mod} * \text{index}$; 输出 =

$\sin(\text{载波相位})$ 。 index 控制调制深度——越大谐波越多(音色越亮)。 $\text{ratio} = \text{调制器频率} / \text{载波频率}$ —— $\text{ratio} = 2$ 产生奇次谐波为主(像单簧管), $\text{ratio} = 3$ 产生更复杂的谐波排列。

为什么是对相位做加法而不是直接改频率?

因为"调相"在数学上等于"调频"——频率是相位的导数,对相位做正弦调制等价于对频率做余弦调制。这和经典的Yamaha DX7合成器完全一样。

3.3 aud_oscillatorbank -- 分频器风琴

模拟Hammond风琴的"分频器"设计。有7个"拉杆"($\text{registration}[0] \sim [6]$),分别控制8英尺、4英尺、2英尺、1英尺四种长度对应的方波和锯齿波。每往上移一档(比如8英尺→4英尺),频率翻倍。

内部做了智能降频:

如果基频太高(比如>采样率/16),会自动把频率降低一个八度并重新分配泛音——保证不会超过奈奎斯特频率。4个 saw_gain 是从 registration 数组计算出来的加权组合。 segment 变量跟踪锯齿波的"整数段",在段边界用BLEP修正。

3.4 aud_formantosc -- 共振峰振荡器

模拟人声/管乐的共振峰。 carrier 频率控制音高, formant 频率控制"音色明亮度"——改变 formant 改变频谱能量的集中位置而不改变音高。 carrier 相位重置时, formant 相位也被重置——用BLEP平滑这个不连续,避免"咔嚓"声。 phase_shift 用一阶惰性平滑(ps_inc 控制逼近速度)。

3.5 aud_variablesawosc -- 可变锯齿波

锯齿波/三角波的连续变形。 $\text{waveshape} = 1 \rightarrow$ 纯锯齿(由 pw 控制斜率), $\text{waveshape} = 0 \rightarrow$ 纯陷波锯齿(有个"缺口")。中间值=两种波形的混合。在 pw 点和周期终点各有一个BLEP修正。频率上限 $0.25 * \text{采样率}$ 。

3.6 aud_variableshapeosc -- 可变波形振荡器

三角→锯齿→方波的连续过渡。 $\text{waveshape} < 0.5$: 三角主导; $\text{waveshape} > 0.5$:

方波主导。支持 sync (同步)功能——主振荡器控制从振荡器的相位重置,产生类似Roland TR系列鼓机的" sync sweep "音色。 $\text{ComputeNaiveSample}$ 做三波形混合: $\text{saw} + (\text{square} - \text{saw}) * \text{sq_amt} + (\text{triangle} - \text{saw}) * \text{tri_amt}$ 。

3.7 aud_vosim -- VOSIM振荡器

两个正弦波(共振峰)被载波同步。载波= $\sin(\text{载波相位} * 0.5 + 0.25) + 1$ (产生一个 $0 \sim 2$ 的包络)。 reset_phase 由 carrier_shape 控制——改变 carrier_shape 改变频谱"尖锐度"。两个共振峰频率独立于载波——改变共振峰改变音色而不改变音高。

3.8 aud_zoscillator -- Z振荡器

类似VOSIM但用更复杂的Z函数。Z函数有三个区段: $\text{mode} < 0.333$ 时纯相位偏移; mode

$0.333 \sim 0.666$ 时相位偏移+ offset 混合;

$\text{mode} > 0.666$ 时纯 offset 。 $\text{discontinuity_phase}$ 用2倍 carrier 频率——每个 carrier 周期产生两个不连续点,各用BLEP修正。

3.9 aud_blosc -- 带限振荡器(LGPL)

用BLIT-BLEP方法的带限振荡器。BLIT = BandLimited Impulse

Train(带限脉冲序列)——用 sinc 函数加权的一系列脉冲。BLEP =

用多项式近似BLIT来降低计算量。内部有4096元素的 vec2 缓冲存储BLEP修正历史。三种波形各自独立实现(ProcessSquare/Triangle/Saw)。

第4章 噪声发生器（9个模块）

4.1 aud_whitenoise -- 白噪声

32位线性同余生成器(LCG)。公式: $X = (1664525 * X + 1013904223) \text{ 对 } 2^{32} \text{ 取余}$ 。这是Numerical Recipes推荐的标准参数,周期为 2^{32} 次(约43亿个采样点后才重复)。输出 = $(X \text{ 右移 } 1 \text{ 位}) / 2147483648.0 - 1.0$, 范围 $[-1, +1]$ 。种子相同则噪声序列相同(确定性)。

4.2 aud_clockednoise -- 时钟噪声

用BLEP平滑的采样保持噪声。frequency参数控制采样的速率——低频=缓慢的随机阶跃,高频=密集的噪声。当frequency>0.25时,额外混入原始白噪声(raw_amount=4*(freq-0.25))产生更明亮的噪声。在采样点变化时用BLEP修正——消除S/H阶跃的混叠。

4.3 aud_dust -- 尘埃脉冲

随机脉冲发生器。每采样点取随机数u,如果u<density就输出u/density,否则输出0。SetDensity(f)内部乘0.3——所以你设0~1对应实际密度0~0.3。低密度=稀疏的"啪嗒"声,高密度=密集的"沙沙"声。纯头文件模块,所有函数static inline。

4.4 aud_particle -- 粒子噪声

把随机脉冲送进SVF滤波器——每触发一次,用随机值重新设置SVF的频率和谐振,模拟"粒子碰撞"。density控制触发概率, sync强制触发。pre_gain(前置增益)根据滤波器的Q值、频率、密度动态调整——保证平均输出幅度稳定。

4.5 aud_fractal_noise -- 分形噪声

3层时钟噪声叠加。底层=最低频率,中层=频率翻倍+增益衰减,顶层=再翻倍+再衰减。所以低频分量大,高频分量小——频谱呈现粉红噪声般的滚降。SetColor控制衰减系数,SetFreq控制最底层的频率。

4.6 aud_grainlet -- 粒子振荡器

对正弦波做非线性相位扭曲。Carrier函数:

shape<1时加速相位的前半段(产生"尖端"),shape>1时产生"破折点"。Grainlet输出=carrier*(formant+bleed)/(1+bleed)——formant是另一个正弦波,bleed控制formant的渗入量。

4.7 aud_smooth_random -- 平滑随机

用smoothstep插值的随机游走。frequency控制游走速度——每1/frequency秒取一个新的随机目标。输出在旧目标和新目标之间做平滑过渡,用的是 $\text{phase} * \text{phase} * (3 - 2 * \text{phase})$

这个smoothstep曲线(常用于游戏动画的缓动函数)。纯头文件模块。

4.8 aud_samplehold -- 采样/跟踪保持

两种模式:

采样保持(触发瞬间抓拍当前输入并保持)和跟踪保持(触发期间持续跟随输入)。两种模式同时运行,根据mode参数选择返回哪个。previous记录上一拍的trigger值做上升沿检测。纯头文件模块。

4.9 aud_maytrig -- 概率触发

最简单的模块: $\text{rand}() / \text{RAND_MAX} \leq \text{prob}$

时返回true。prob=1永远true,prob=0永远false。用于随机触发鼓声或调制。纯头文件模块。

第5章 动态处理（4个模块）

5.1 aud_limiter -- 峰值限幅器

来自Mutable Instruments的STMLib。用SLOPE宏做非对称包络跟随：

上升速度0.05(快速响应峰值), 下降速度0.00002(极慢释放防失真)。gain = (peak<=1)? 1 :

1/peak(将信号缩回安全范围)。最后过SoftLimit做软剪。就地修改输入缓冲, 不分配输出缓冲。

5.2 aud_compressor -- 压缩器(LGPL)

基于Faust算法。threshold(阈值dB)、ratio(压缩比, 比如4:1)、attack(起音秒数)、release(释音秒数)四个核心参数。

内部用exp系数做包络跟随——attack越快系数越大, release越慢系数越小。gain_rec用长时平滑(atk_slo2)追踪信号电平, 在log域计算需要的衰减量。makeup_gain补偿压缩造成的音量降低——自动模式:

|thresh-thresh/ratio|*0.5。旁链(sidechain): Process(in, key)用key信号控制压缩量而非用in自控。

5.3 aud_crossfade -- 交叉淡变

在A和B两个信号之间平滑过渡。四种曲线:

线性(均匀过渡)、恒功率(用sin/cos保证 A^2+B^2 不变——DJ混音关键)、对数(匹配人耳响度感知)、指数(快速切换)。pos=0→全A, pos=1→全B, pos=0.5→居中。

5.4 aud_balance -- 信号平衡(LGPL)

将sig的响度匹配到comp的响度。分别对sig²和comp²做一阶低通滤波(模拟RMS电平检测), 得近似功率。sqrt(r/q)得增益比, 乘到sig上。ihp(内部高通频率)越低平滑越慢但越稳。

第6章 效果器（13个模块）

6.1 aud_chorus -- 合唱效果

双声道合唱。两个独立的ChorusEngine(左/右), 每个包含: 三角LFO + 分数延时线 +

反馈。LFO相位来回扫(phase正向走, 超限折返并反转频率方向), 产生来回摇摆的延时——模拟多位演奏者演奏同一音符时的"时间差"。延时范围0.1~8ms(合唱典型)。pan控制左右分布。GetLeft/GetRight取声道。

6.2 aud_flanger -- 镶边效果

类似合唱但延时更短(0.1~7ms)反馈更大(>0.9)。短延时使梳状滤波的"齿"间距变宽——产生标志性的"喷气飞机"声。LFO结构同合唱。

6.3 aud Phaser -- 移相效果

8个全通级联。每个全通用延时线实现($y = -coeff * write + read$)。LFO调制全通的截止频率, 产生"频率相关的相位抵消"。os=30Hz偏移防止极低频率的crunch音。外层Phaser类封装8个PhaserEngine, 输出为8路之和。

6.4 aud_tremolo -- 颤音效果

用振荡器(支持全部8种波形)调制音量。modsig = dc_os + osc输出; output = input * modsig。dc_os = 1-depth确保调制围绕1摆动(不是从0到2*深度)。正弦LFO=经典颤音, 方波LFO=门控"断音"。

6.5 aud_overdrive -- 过载效果

SoftClip失真+前后增益。drive参数控制失真量:

pre_gain在低drive时几乎线性, 高drive时急速增大(drive^5项主导)。post_gain补偿失真造成的信号衰减。SoftClip在±3内用三次有理函数做软过渡, 超出硬限。

6.6 aud_wavefolder -- 波形折叠

foldback失真: 信号超过1时不是被剪掉而是被"折回"——超过越多折回越多次。ft = floor((in+1)*0.5) 计算折叠次数, sgn = (int)ft奇偶决定方向, 输出 = sgn*(in-2*ft)。支持负增益实现through-zero折叠。

6.7 aud_autowah -- 自动哇音

Soundpipe算法。先用整流+两级一阶LP做包络跟随检测输入信号的"响度"。响度控制一个IIR滤波器的共振频率——响度大时频率高, 产生"哇"的效果。6个内部状态(rec0~rec5)存储滤波器历史。wah和drywet分别控制效果和干信号混合比。

6.8 aud_pitchshifter -- 移调器

基于延时线扫描。延时匀速变化产生多普勒效应=频率偏移。shift为半音数, ratio=2^(shift/12)。feedback制造多重移调。LFO调制延时产生合唱化效果。mix控制干湿比。简化单延时线实现(非PSOLA/PVS), 极限移调时音质有限。

6.9 aud_decimator -- 降采样+Bitcrush

两级处理。1)降采样:

threshold=(factor^2)*96, 内部计数器inc每到threshold时更新downsampled=当前输入(模拟降低采样率的S/H效应)。2)Bitcrush:

*65536放大->右移bits位丢弃低位->左移用0填充->/65536还原。smooth模式用bit_overflow做非整数位深的"软bitcrush"。

6.10 aud_sampleratereducer -- BLEP降采样

用BLEP平滑的降采样。frequency=输出采样率(0~1归一化)。相位每超1时:

用线性插值重采样, 再用BLEP修正不连续——所以降低采样率不产生混叠(因为BLEP把高频去掉了)。previous_sample跟踪输入, 供插值使用。

6.11 aud_bitcrush -- Bitcrush+折叠(LGPL)

先做bit深度缩减(乘65536->偏移->*bits/65536->floor截断->*65536/bits->去偏移->/65536), 然后送Fold模块做后续降采样。fold增量=采样率/crush_rate。

6.12 aud_fold -- 折叠(LGPL)

用可调速率的S/H实现折叠: `index < sample_index`时连续更新(`value=in`); 否则hold上一个值。`incr`控制折叠速率。速率高时接近bitcrush, 速率低时接近随机步进。

6.13 aud_reverb -- FDN混响(LGPL)

8条反馈延迟网络(FDN)模拟房间混响。每条线有独立的延迟时间和反馈系数——从预优化参数表取值。内部用32位定点小数(读位置28位移位)做高精度延时。`Cubic`(4点Hermite)插值平滑读取。种子驱动的随机线段发生器(每段随机的延迟和持续时间)模拟真实房间的不规则反射。低通阻尼(`damp_fact`)模拟高频吸收。~395KB `aux`缓冲。

第7章 物理建模（7个模块）

7.1 aud_pluck -- Karplus-Strong拨弦

声音 = 噪声在延时线里循环衰减。初始化时用随机噪声填充缓冲区(模拟拨弦瞬间的激励), 然后每个采样点: 读一个值->滤波衰减->写回缓冲的另一端。就像弦被拨后, 振动在弦上来回反射, 每次反射能量减少一点。

两种衰减模式:

递归(每个新值=(当前+前一个)*coeff, 经典的K-S)和加权平均(新值=当前*coeff+前一个*(1-coeff), 衰减更长)。coeff由damp参数控制: damp高=快衰减=尼龙弦;

damp低=长延音=钢弦。频率控制相位增量和缓冲长度。phs256用8位小数做线性插值。

7.2 aud_karplusstring -- K-S弦(扩展)

在标准K-S之上增加:

分数延时线(更精确的频率控制)、DC阻断(防止低频能量累积导致输出漂移)、一阶滤波(模拟弦上高频在不同张力下的传播差异)、非线性(nonlinearity参数的立方加到延时线输出, 模拟弦的大幅振动张力)。

7.3 aud_stringvoice -- 弦乐音色(Rings风格)

完整弦乐:

激励(噪声或脉冲)->激励滤波器(SVF低通)->K-S弦。sustain模式用Dust持续激励(模仿弓弦持续摩擦), 非sustain用单次脉冲(拨弦)。brightness同时控制激励滤波截止和dust密度——brightness高=亮音+密集噪声;

brightness低=暗音+稀疏噪声。damping控制弦衰减速度。aux输出激励信号(用于监测或外部处理)。

7.4 aud_resonator -- 谐振体模拟

24个SVF模式模拟木琴/锣等复杂谐振体。每4个一组做批量处理(一次循环计算4个模式, SIMD友好)。CalcStiff将structure转为刚度系数——低值时谐波间距变窄(倒转), 高值时间距变宽(拉伸)。NthHarmonicCompensation基于刚度计算每个谐波的拉伸因子。brightness控制Q值的谐波间衰减, damping控制总衰减。

7.5 aud_modalvoice -- 模态合成音色

槌子激励->1阶SVF低通->24模式Resonator。sustain模式下用Dust持续激励。aux输出激励信号。想象用木槌敲金属条或木块——激励是短脉冲, 谐振体决定了音色。

7.6 aud_drip -- 水滴声

Perry Cook的经典模型(2000年)。3个并联二阶谐振器(模拟水滴落下的三种频率)。随机抖动:

用my_random以概率触发新的频率"闪烁"——模拟水滴落下时不同部分的振动。3级级联产生短暂回声。shake能量模仿水滴的振幅抖动和自然衰减。输出=0.005*(finalZ2-finalZ0)——三级差分乘以极小增益。

7.7 aud_polypluck -- 复音拨弦

最多4声部Pluck封装。每声部独立频率/衰减/阻尼, 外加DC

Block。第一声部接收外部触发, 其余声部不触发(由第一声部的谐波关系决定频率)。输出=全部声部求和/声部数。

第8章 鼓合成（5个模块）

8.1 aud_analogbassdrum -- 808底鼓

两个脉冲发生器：

主脉冲（幅度 $3+7*\text{accent}$ ，持续1ms）模拟"击打感"，FM脉冲（持续6ms）控制"音高扫变"。Diode函数模拟二极管的非线性——正半周直通，负半周被 $0.7*x/(1+|x|)$ 压缩。

主脉冲→1极LP（0.1ms时间常数）→Diode→产生"click"。FM脉冲→LP→乘 $\text{attack_fm}+\text{self_fm}$ （两个FM量控制chirp深度）→改变谐振器（SVF）的频率→Band/Low输出。tone_lp做最终色调控制。sustain模式旁路谐振器，直接正弦波+LP输出持续低音。

8.2 aud_analogsnaredrum -- 808军鼓

5个频率模式（1.00/2.00/3.18/4.16/5.62倍基频）。tone<0.667时只用前2个模式（原版808的简单设计），tone>=0.667启用全部5个（扩展设计——更像真实军鼓的多模式）。脉冲驱动谐振器产生"腔体"声，噪声+SVF带通产生"沙沙"声。snappy控制两部分的混合比。pulse_lp用fclamp做一阶平滑。

8.3 aud_synthbassdrum -- 合成底鼓（909风格）

三个子模块：Click（脉冲→SVF 5000Hz

LP，用SLOPE非线性产生"脆响"）、AttackNoise（噪声→LP0.05-HP0.005=窄带噪声）、主鼓（DistortedSine+TransistorVCA）。

DistortedSine：

三角波转正弦→sinf版本的正弦。dirtiness在两者间插值——0=纯正弦，1=三角失真。TransistorVCA模拟晶体管饱和： $s=(s-0.6)*\text{gain}$ ，return $3*s/(2+|s|)+\text{gain}*0.3$ 。体包络（Body env）控制持续音，瞬态包络（Transient env）驱动click+noise。

8.4 aud_synthsnaredrum -- 合成军鼓

两个失真正弦（freq和freq*1.47，模拟909军鼓双模式）。reset_noise模拟909电路中的互调：

phase>0.5?方波+1:-1，乘rna*0.025加回相位重置。两路正弦和（0.60+0.25权重）→LP→drum通道。白噪声→LP→HP→(snare+f0)*level→snare通道。snappy混合两通道。hold_counter（40~70ms）模拟军鼓响弦的机械延迟。

8.5 aud_hihat -- 808踩镲

6个施密特方波振荡器（频率比1.0/1.304/1.466/1.787/1.932/2.536）。用32位定点相位（4294967296=1.0），相位最高位(>>31)提取方波值。6个输出求和/6。噪声路径：

BPF（SVF，频率跟随tone）+noisiness*clk_noise混合。VCA（Linear或Swing模式）。HPF输出。envelope用SemitonesToRatio指数衰减。sustain下关闭envelope衰减（持续输出）。

第9章 包络与控制（5个模块）

9.1 aud_adenv -- AD包络

两段包络：

Attack (从触发时的值升到1), Decay (从1降到0), IDLE (输出0)。触发后current_segment=ATTACK, phase从0开始。Attack段end=1, Decay段end=0。phase每采样+1, 经历time_samps个采样后完成。

curve_scalar控制曲线：

=0线性(匀速), 不等0指数曲线(先快后慢, 用10次乘法迭代的expf_fast代替库expf)。segment过渡: out>=1且正在Attack -> 进Decay; out<=0且正在Decay -> 进IDLE。最终输出 = out*(max-min)+min, 可调输出范围。

9.2 aud_adsr -- ADSR包络

指数衰减型四段包络。attackShape用 $9 * \text{pow}(\text{shape}, 10) + 0.3 * \text{shape} + 1.01$ ——shape=0时target~1.01(几乎瞬时攻击), shape=1时target~10.3(大幅overshoot后回落到1——产生"冲击感")。

D0系数: $\text{attackD0} = 1 - \exp(\log(1-1/\text{target})/(\text{time} * \text{sr}))$ 。decayD0/releaseD0: $\text{D0} = 1 -$

$\exp(\log(1/e)/(\text{time} * \text{sr}))$ 。gate触发(不是trigger脉冲): gate上升沿->Attack; gate保持高->Decay->Sustain;

gate下降沿->Release。Sustain level<=0强制IDLE。

9.3 aud_line -- 线性渐变

start->end在duration秒内线性过渡。每采样+inc=(end-start)/(sr*duration)。到底后finished=1并保持end值。Start随时重新触发新渐变。

9.4 aud_port -- 滑音

一阶LP平滑: $y += c1 * (\text{in} - y)$ 。c1 =

$1 - \exp(-\ln(2)/(\text{htime} * \text{sr}))$ 。半衰期=htime秒——经过htime秒输出到达目标值的一半。指数趋近——刚变时快, 接近目标时慢。

9.5 aud_metro -- 节拍器

固定频率脉冲。phase每采样+phase_inc, >=1时减1并返回true, 否则false。即每1/freq秒一个上升沿。

第10章 采样与循环 (2个模块)

10.1 aud_granularplayer -- 粒子采样器

双粒子引擎。每个粒子:

采样的一个短片段(由`grain_size`决定), 用256点余弦包络做加窗(起始和结尾淡入淡出, 消除衔接杂音)。 `phs/phs2`控制读取位置(正反向), `phs1mp/phs1mp2`控制移调量(通过改变读取位置的前进速度)。 `NegativeInvert`处理负频率(正向时输出读取位置, 负向时输出反相位置)。 `WrapIdx`处理采样数组的环回。 `CentsToRatio`: 音分转频率比。

10.2 aud_looper -- 循环录音

四种模式:

NORMAL (无限叠加)、**ONETIME_DUB** (只叠加一次)、**REPLACE** (替换录音)、**FRIPPERTRONICS** (每次循环*0.707衰减, 模拟磁带延迟)。状态机:

EMPTY→**REC_FIRST**→**PLAYING**→**REC_DUB**。 `trigger`切换状态。1200采样交叉淡变窗口($\sin(\pi/2 * pos)$ 加权)消除循环首尾衔接的"啪"声。半速/反向分别改变播放速度(`inc=0.5/inc=-1`)。 `rec_queue`延迟**ONETIME_DUB**到循环边界。

第11章 工具（2个模块）

11.1 aud_jitter -- 抖动调制器

随机分段线性调制源。32位定点相位 ($0 \times 1000000 = 1.0$)。每段：

cps在min~max间随机, 起点num1随机, 终点num2随机。段内线性过渡 ($dfd_max = (num2 - num1) / FT_MAXLEN$)。输出 $= (num1 + phs * dfd_max) * amp$ 。段结束 ($phs \geq FT_MAXLEN$) 时重新随机cps/num1/num2。产生的信号是一个"slow random walk"——段间平滑, 段内线性。适合模拟电路元件漂移等缓慢随机调制。

11.2 aud_pitchshifter -- 移调器

延时线扫描移调。ratio=powf(2, shift/12)——shift正=升调, shift负=降调。LFO调制延时量产生合唱化的移调效果。feedback控制反馈量产生多重移调叠加。mix控制干湿比。简化实现——单延时线, 非PSOLA, 极端的移调量下音质会下降。

附录 全部66模块分类索引

核心工具

aud_dsp aud_phasor aud_dcblock aud_delayline

滤波器

aud_svf aud_onepole aud_tone aud_allpass aud_comb aud_moogladder aud_mode aud_nlfilt aud_soap

振荡器

aud_osc aud_fm2 aud_oscillatorbank aud_formantosc aud_variablesawosc aud_variableshapeosc aud_vosim
aud_zoscillator aud_blosc aud_grainlet aud_harmonic_osc

噪声

aud_whitenoise aud_clockednoise aud_dust aud_particle aud_fractal_noise aud_smooth_random aud_samplehold
aud_maytrig

动态

aud_limiter aud_compressor aud_crossfade aud_balance

效果器

aud_chorus aud_flanger aud Phaser aud_tremolo aud_overdrive aud_wavefolder aud_autowah aud_pitchshifter
aud_decimator aud_sampleratereducer aud_bitcrush aud_fold aud_reverb

物理建模

aud_pluck aud_karplusstring aud_stringvoice aud_resonator aud_modalvoice aud_drip aud_polypluck

鼓

aud_analogbassdrum aud_analogsnaredrum aud_synthbassdrum aud_synthsnaledrum aud_hihat

包络

aud_adenv aud_adsr aud_line aud_port aud_metro

采样

aud_granularplayer aud_looper

工具

aud_jitter aud_pitchshifter

本手册基于AUDIO_DSP库v1.0。原始代码版权归Electrosmith Corp及各原作者所有(MIT/LGPL协议)。手册采用CC-BY自由使用。